Mathematical Induction

Mathematical induction is a proof technique for showing that some property exists for a sequence of expressions. Mathematical induction is a two-step process. First, you must show that the property exists for the first element in the sequence. Second, you assume that the property holds for the *k*th element in the sequence and then using proof techniques show that it holds for the *(k+1)*st element in the sequence. Written formally,

$$(P(1) \wedge \forall k\,(P(k) \rightarrow P(k+1)) \rightarrow \forall n\,P(n)$$

This formally says what is written in words above. If the property holds for the first element, and assuming that it holds for an arbitrary element, you can show that it holds for the subsequent element. Then you have proved by induction that the property holds for all elements.

Induction is a useful technique for proving formulas involving sums of numbers. For example, using induction, you can show that

$$1+2+\cdots+n=\frac{n(n+1)}{2}$$

Induction can also be used to show certain inequalities hold. For example, for all positive integers n, $n<2^n$.

Observe what is similar in these examples: there is an integer variable, and you are asked to prove the expression is true for all values of that integer variable. This is an important characteristic of induction.

Strong Induction and Well-Ordering

Strong induction is a variation mathematical induction. The basis step in strong induction is the same. However, the inductive step differs. In strong induction, you assume that the property in question holds for all values $1,\ldots,k$ and then prove that it holds for the $(k+1)$ st value.

Mathematical induction and strong induction can all be shown to follow from a fundamental axiom on the set of integers, the well-ordering property of integers. Essentially, the well-ordering property says that every non-empty set of integers has a least element.

Recursive Definitions and Structural Induction

Mathematical induction and strong induction are interesting from a mathematical point of view. Their role in computer science leads to recursively defined functions. Recursive functions are specified by giving a value for the function at zero and giving a rule for finding a value at an integer from values at smaller integers. A classic example is the factorial function $n! = 1 \cdot 2 \cdot \ldots \cdot n$. A recursive definition of this function is

$$0! = 1$$
$$n! = n(n-1)!$$

The first line provides the basis step, and the second line shows how to compute the factorial using the value of the factorial at smaller integer values of the factorial function.

Recursion may be used to define other such as sets and other structures. For example, consider the subset S of integers defined by

$$2 \in S$$
$$x, y \in S \rightarrow x \cdot y \in S$$

The elements of S are found by first noting that 2 is in the set. Next, 2*2=4 is in the set. Then 2*4=8 and 4*4=16 is in the set. Continuing in this fashion, note that the set contains the all powers of 2.

Given an alphabet of characters, you can also recursively define all the character strings in that alphabet. A recursive definition can also be used to define rooted trees. If you limit the number of vertices in a tree to two, then the rooted trees are binary trees.

You can also define well-formed formula of operators and operands using recursion. Suppose that set of well-formed formula consists of variables, numerals, and the binary operators $+, -, *, /, \wedge$. Here * means multiply, and $\wedge$ means exponentiation. The basis step in the definition is that x is a well-formed formula if it is a numeral or a variable. The recursive step says that if F and. .are well-formed formula, then $(F+G)$, $(F-G)$, $(F*G)$, (F/G), and $(F \wedge G)$ are well-formed formula.

Recursive Algorithms

An algorithm is said to be recursive if it solves a problem by reducing it to the same problem with a smaller input. The factorial problem has a recursive definition.

procedure $factorial(n : \text{nonnegative integer})$
    if $n = 0$ **then return** 1
    else return $n \cdot factorial(n-1)$

Observe that this procedure calls itself with a smaller value of the argument input to the procedure. Because the input to the procedure is a nonnegative integer, successively subtracting one from the input must eventually lead to a condition for which the input to the procedure is equal to zero.

There are recursive algorithms for computing nonnegative integer powers of nonzero real numbers, for computing the greatest common divisors, and many other computations. The linear search and binary search algorithms discussed earlier also have recursive algorithms for their solutions.
Mathematical induction and strong induction are the techniques to use in order to prove the correctness of recursive algorithm.